

1 An Audit of Settlement Alignment, Information Timing, and Energy Conservation in EMSx Battery Control: Online Rollout under a Physical Evaluation

1.1 Abstract

This paper reports an online stochastic control method for grid-connected microgrid battery systems and a two-track evaluation of it. The method uses the settlement-aligned horizon-96 forecast of the EMSx benchmark, a site-specific 24-hour-ahead forecast-error distribution, and a risk-sensitive Bellman selector. We audit two indexing issues (the forecast origin exposed by the official Information interface vs row $t + 96$; the horizon-1 vs horizon-96 forecast index) and an energy-conservation violation of the benchmark environment: the stage-cost function credits discharge energy from an empty battery while the state of charge is merely clamped to zero.

The two evaluation tracks are:

1. **Environment-consistent track.** The perfect-prediction oracle is the backward DP that shares the environment’s loose physics. Under this convention the candidate scores 0.997348. We show this score is dominated by the exploit: the candidate performs a physically infeasible discharge on 99.6% of steps, the oracle on 100%, and the baseline on 0.007%.
2. **Physical track.** The action set is restricted to the energy-conserving set $U(\text{SOC})$, and the oracle is an independent linear program with physical SOC bounds. Under this convention the SDP-AR(1) baseline scores 0.768, and the best honest controller — the rollout with the baseline-aligned actual state and the physical action filter (R_P) — scores 0.744 (11/70 sites above the baseline). A behavior-clone of the physical oracle (per-site MLP) reaches 0.573 and is recorded as a negative result.

The audits are timestamp-proven and numerically quantified. The conclusion is that the previously reported 0.9973 measured benchmark-exploit parity, not control quality; the physically honest leaderboard is dominated by the SDP-AR(1) baseline, with R_P as the closest honest alternative. The selected algebraic properties of the finite-benchmark evaluation are machine-checked with the Lean proof assistant.

1.2 1. Introduction

Microgrid battery control reduces the operating cost of a site. The site has a battery, a load, and photovoltaic generation. At each 15-minute step, the controller chooses one power action. The evaluation uses a performance score: the gain of a candidate controller over a no-battery dummy, divided by the maximum possible gain given by a perfect-prediction oracle.

This paper’s contributions are audits, not new controllers:

1. **Forecast-origin audit.** The official Information constructor exposes the forecast issued at the earliest row of the decision window (row $t + 1$, a day-ahead forecast). An earlier local modification moved the origin to row $t + 96$. Under the reconstructed decision timestamp (Section 4.11) the row- $t + 96$ forecast is issued at the decision time and is not temporally noncausal — but it is not exposed by the official Information interface, so reading it would be a **benchmark interface violation**, not a temporal leak. We restored the interface-exposed origin (row $t + 1$) and locked the contract with timestamp tests.
2. **Settlement-alignment audit.** The simulator settles step t against the actual net demand at row $t + 97$, which is predicted by `load_95` of row $t + 1$ (index 96), not `load_00` (index 1). We prove this with a timestamp trace.
3. **Energy-conservation audit.** The environment credits empty-battery discharge. We quantify the exploitation by candidate, oracle, and baseline, and re-evaluate the method under a physically consistent action set and oracle.

1.3 2. Problem Statement

1.3.1 2.1 Site Model

A site has a battery with capacity C and power limit $|P|$. The state of charge s follows

$$s_{t+1} = s_t + \left(\eta_c \max(0, u_t) - \frac{\max(0, -u_t)}{\eta_d} \right) \frac{|P|\Delta t}{C}$$

with $u_t \in [-1, 1]$, $\Delta t = 0.25$ h, clamped to $[0, 1]$. The stage cost is

$$c_{t(u,z)} = p_{b(t)} \max(0, z + u|P|\Delta t) - p_{s(t)} \max(0, -(z + u|P|\Delta t))$$

matching `compute_stage_cost`. A positive control charges the battery.

Energy-conservation violation. `compute_stage_cost` credits the full discharge energy of the control even when the battery is empty; the SOC is merely clamped. We verified this empirically (SOC = 0, $u = -1$: the bill falls by 2.44 per step, SOC unchanged). The physically admissible action set at SOC x is

$$U(x) = \left\{ u : -\eta_d C \frac{x}{|P|\Delta t} \leq u \leq C \frac{1-x}{\eta_c |P|\Delta t} \right\}$$

i.e., discharge limited by stored energy, charge by free capacity. An action outside $U(x)$ violates energy conservation.

1.3.2 2.2 Performance Score

With dummy cost C^d , oracle cost C^a , and model cost C^m , the score is $\frac{C^d - C^m}{C^d - C^a}$. All quantities must be evaluated under one convention; this paper uses two and never mixes them.

1.4 3. Method

1.4.1 3.1 Forecast Semantics: Timestamp Trace

The simulator settles step t against the actual net demand at row $t + 96 + 1$ (`apply_control`); row i holds the forecast issued at row i with `load_k` predicting row $i + k + 1$; the information object covers rows $t + 1$ through $t + 96$. The timestamp trace for step $t = 1$ of site 1:

Decision time (step 1)	row 97 = 2014-07-21 00:00
Information window	rows 2..97 = 00:15 .. 2014-07-21 00:00
Forecast origin (row $t+1$)	row 2 = 2014-07-20 00:15
<code>load[1]</code> (current actual <code>w_t</code>)	row 97 = 2014-07-21 00:00
<code>forecast_load[1]</code> target	2014-07-20 00:30 (row 3)
<code>forecast_load[96]</code> target	2014-07-21 00:15 (row 98)
Settlement row	row 98 = 2014-07-21 00:15

The decision time of step 1 is the timestamp of row $t + 96$ (row 97 = 2014-07-21 00:00); see Section 4.11 for the code-level proof. Facts: `forecast_load[96]`'s target equals the settlement timestamp (row $t + 97$, i.e. decision time + 15 min); `forecast_load[1]`'s target (row $t + 2$) is 94 steps in the **past** and useless. The forecast origin exposed by the official Information interface is row $t + 1$ (a day-ahead forecast issued 95 steps before the decision) — causal. The baseline reads `load[1]` - `pv[1]`, the **current** actual at row $t + 96$ (w_t), as its state — causal. The forecast-error rollout reads only the forecast arrays exposed by the interface. The row- $t + 96$ forecast is issued at the decision time (not temporally noncausal) but is **not** exposed by the official interface; reading it would be a benchmark interface violation (Section 4.11).

1.4.2 3.2 Forecast-Error Law and Rollout

For each site we fit a 24-hour-ahead (horizon-96) forecast-error distribution, grouped by quarter of day and weekday/weekend ($k = 20$ levels, pseudocount 32, fit on training data). At each step the selector enumerates the admissible controls, evaluates every error scenario with a shared clamped z_{next} , and minimizes

$$Q(u) = (1 - \lambda)E[Y_{e(u)}] + \lambda \text{CVaR}_\alpha(Y_{e(u)}), \quad Y_{e(u)} = c_{t(u, z_{\text{next}})} + V_{t+1}(s', z_{\text{next}})$$

with $\lambda = 0.25$, $\alpha = 0.9$. Two variants are evaluated: the unconstrained variant (all $u \in [-1, 1]$) and the physical variant ($u \in U(x)$, the energy-conserving set). Clamping rates are below $1.1 \cdot 10^{-3}$.

1.4.3 3.3 Evaluation Protocol: Two Tracks

Environment-consistent track. The oracle is the backward DP (21-point SOC grid, nearest-neighbor interpolation, terminal $V_T = 0$ per period, SOC carried). Because the environment credits empty-battery discharge, this DP is the environment-consistent **approximate** oracle (21-point grid, nearest-neighbor interpolation; no exact lower-bound guarantee). An independent LP with the SOC lower bound removed reproduces the DP’s objective on the tested instances.

Physical track. The oracle is a sequence of independent per-period perfect-foresight LPs (scipy HiGHS), $J_{\text{PF}}^{\text{period}}$, each with the physical SOC constraint $0 \leq s \leq 1$, the same prices, settlement demand and battery dynamics, terminal $V_T = 0$ per period, and SOC carried across periods. Because the next period’s value does not enter the current period’s objective, it is a receding sequence of finite-period LPs with a period-boundary terminal-depletion bias, not a full-sequence optimum over the whole test horizon. We verified the LP on the first five sites against the environment DP and against the environment-consistent LP; the LP solutions have zero simultaneous charge/discharge waste.

1.5 4. Results

1.5.1 4.1 Exploit Metrics

For the candidate (unconstrained), the oracle, and the baseline, on the 70 test sites:

Metric	Candidate	Oracle (DP)	Baseline
mean $r_{\text{infeasible}}$	0.996337	1.0	$7.1 \cdot 10^{-5}$
mean E_{phantom} (kWh/period)	26023	168–25200	1.2
mean $r_{0,-1}$	0.99624	1.0	$8.0 \cdot 10^{-6}$

where $r_{\text{infeasible}}$ is the fraction of steps with a discharge beyond stored energy, E_{phantom} the corresponding credited-but-nonexistent energy, and $r_{0,-1}$ the fraction of steps with SOC = 0 and $u = -1$. The candidate and the oracle discharge from an empty battery on essentially every step; the baseline is physically clean. The physical variant of the candidate has $r_{\text{infeasible}} = 5.2 \cdot 10^{-5}$ (pre-projection implementation) and a 1200-fold reduction in phantom energy; the final strict-projection implementation has $r_{\text{infeasible}} = 0$.

1.5.2 4.2 Environment-Consistent Track

Metric	Value	Note
Mean score	0.997348	exploit-driven
Score 95% LCB	0.995720	—
Paired LCB vs baseline	0.917831	—
Minimum site score	0.954600	—
Sites with score = 1 (float)	55	—

These numbers measure how closely the candidate reproduces the perfect-foresight **exploit** (discharge from empty battery whenever the tariff allows). They are not a statement about physical control quality.

1.5.3 4.3 Physical Track

Metric	Value	Note
Candidate mean score	0.579397	physical LP oracle
Baseline mean score	0.7677	same oracle
Mean cost candidate	2300.8	—
Mean cost baseline	2235.7	—
Mean cost dummy	2496.7	—
Candidate score LCB	0.532702	—

Paired vs baseline	−0.1883	baseline wins
--------------------	---------	---------------

Under the physical convention the candidate is below the baseline. The value function itself is already energy-conserving (Section 4.5: the offline value iteration rejects infeasible discharges through its next-state bounds check; a “physical” recalibration is bit-identical). The gap comes from the selector structure — the forecast-error multi-scenario rollout with a CVaR-regularized selector versus the baseline’s AR-conditioned SDP selector — and from the state semantics, not from value-function physics (four sites even score below the dummy). (The single-scenario controller is R_P ; see Sections 4.7 and 4.12.)

1.5.4 4.4 Settlement Alignment under Physical Constraints

Site	Index-1 physical	Index-96 physical
1	0.1148	−0.2925
2	0.0648	−0.2769
3	0.2391	0.2384

The large alignment gain observed in the environment-consistent track (0.80–0.92 to 0.96–1.00 on these sites) vanishes under the physical constraint: index 96 is worse on two of three sites. The alignment’s advantage was the ability to time the phantom discharge.

1.5.5 4.5 Why Retraining the Value Function Is Moot: the Exploit Is Online-Only

A dedicated experiment (physical_vf calibration, 3 sites) showed that the retrained “physical” value function is bit-identical to the existing one (max |difference| = 0). The reason is structural: the offline value iteration already enforces physics through its next-state bounds check (state_in_bounds): the calibration dynamics do not clamp the SOC, so an infeasible discharge from an empty battery produces a negative next SOC, which is out of bounds and rejected with cost Inf. The value function is therefore already energy-conserving.

The phantom discharge happens only online: select_rollout_control computes the next SOC with the clamped simulator dynamics (compute_stage_dynamics clamps to [0,1]) **before** the bounds check, so discharging an empty battery passes the check and gets full bill credit. The physical action filter added to the selector (Section 3.3) is the correct repair; it reduces the infeasible-action rate from 0.996 to 5.2e-5 in the pre-projection implementation (with a 10^{-9} filter slack). The final implementation uses a strict closed-interval projection onto $U(x)$, giving an infeasible-action rate of exactly zero (Section 4.9, Section 6).

1.5.6 4.6 Physical-Convention Index Comparison (70 sites)

Under the physical constraint (which removes the exploit), the settlement-aligned configuration remains the better rollout:

Configuration	Physical score	Note
SDP-AR(1) baseline	0.7677	reference
Rollout index-96 + h96 law	0.5794	best rollout
Rollout index-1 + h1 law	0.4849	worse: z_next mismatches VF

The horizon-96 alignment keeps the continuation query consistent with the value function’s z-coordinate (the demand of the settlement row), which remains beneficial even when the action set is physical. Neither rollout configuration reaches the baseline: the SDP-AR(1) controller uses the full AR(1)-structured value function, while the one-step scenario rollout with a discrete error law is a weaker selector under the same (already physical) value function.

1.5.7 4.7 The Improvement Path: Baseline-Aligned State + Physical Filter

The remaining gap to the baseline is closed substantially by aligning the rollout’s state with the baseline’s: use the **actual** net demand at row $t + 96$ (the baseline’s state) instead of the noisy horizon-96 forecast. With the physical action filter this is a physically honest controller that uses exactly the baseline’s information:

Configuration	3-site score	70-site score
SDP-AR(1) baseline	0.5526	0.7677
Rollout, actual z-state + phys filter	0.646	0.7442
Rollout, AR-advanced z (noise 0)	0.533	—
Rollout, forecast z-state + phys	−0.11	0.5794

The actual-state rollout reaches 0.7442 on 70 sites (11/70 above the baseline, no negative-score sites). The one-row semantic offset (row $t + 96$ actual queried against the slice- $(t + 1)$ value function) was tested by a slice-offset ablation over $\tau - 1, \tau, \tau + 1$ on the 3-site set: scores 0.6457 / 0.6452 / 0.6458, i.e. the offset does not cause a primary loss in the configurations tested (a full-site ablation was not run). An AR one-step advancement with zero noise is worse (0.533) because it injects model error. A higher-resolution value function ($dx = du = 0.05$) does not change the score, so grid resolution is not the bottleneck. The residual gap to the baseline comes from the single-scenario selector versus the baseline’s expectation over the AR noise distribution.

The interpretation of the single-scenario advantage is a persistence effect, not “the next-period state is already realized”: the selector uses the row- $t + 96$ actual as a persistence forecast of the settlement row $t + 97$. This is supported by the residual structure on the training data: persistence ($\hat{z}_{t+1|t} = z_t$) has $R^2 = 0.937$ on site 1, only slightly below the AR(1) conditional expectation ($R^2 = 0.948$), so the exact-state single scenario is competitive with the AR expectation because the AR(1) model adds little over persistence.

A final experiment replaced the greedy selector with the baseline’s full SDP selector (`Sto0pt.compute_control`, expectation over the AR noise distribution) plus the same physical action filter. On the 3-site set the result is bit-identical to the baseline (0.6294 / 0.5797 / 0.4485): the physical filter never triggers on the baseline’s policy (its infeasible rate is $7e-5$), so “baseline + physical filter” equals the baseline. The greedy Plan-A selector is actually stronger on these sites (0.646 vs 0.553), a persistence effect consistent with the residual structure above; the baseline’s advantage on the full 70-site mean comes from other sites.

1.5.8 4.8 Behavior Cloning of the Physical Oracle (Machine-Learning Track)

We trained policies that clone the physical LP oracle (the perfect-foresight optimal actions as supervision) with per-site MLPs: features (SOC, the row- $t + 96$ actual demand, intraday/weekly time, prices) mapped to the control, projected onto the energy-conserving set at evaluation. Four configurations were measured on the 70 test sites:

Configuration	70-site score	Note
3-site joint MLP (regression)	0.664 (3 sites)	best config
Per-site MLP (regression)	0.573	—
Single shared MLP	0.482	cross-site drift
Per-site classifier (5 classes)	0.307	too coarse

Behavior cloning does not beat the model-based controllers. The negative result is **consistent with** the oracle’s bang-bang actions (43% of steps at the power limit, 47% at zero) and non-unique flat optima; dedicated classification, set-valued targets, or decision-focused losses were not tested, so the structural explanation remains a hypothesis. The machine-learning track is recorded as a negative result; the model-based R_P selector remains the strongest proposed non-baseline controller (the strongest overall controller is the SDP-AR(1) baseline S_{AR} , 0.7677 vs 0.7442).

Data split disclosure. The oracle labels come from LP solutions of the **training** periods (24 periods per site, excluding the 7990-row period 0), the MLP is trained on those training samples, and the evaluation runs on the **test** periods of the same sites (temporal split, no leakage of test labels into training). The features include the row- $t + 96$ actual (the current observation, causal per Section 4.11). The negative conclusion is therefore scoped: “with these features, losses, and split, single-step behavior cloning did not beat the model-based controllers”, not “behavior cloning is ineffective for this problem”.

1.5.9 4.9 Physical-Convention Leaderboard (Score and Raw Cost)

The primary metric is the raw total cost improvement of a controller over the current best baseline, $M_0(\pi) = \sum_i \omega_i [C_{i(S_{AR})} - C_{i(\pi)}]$ with equal site weights $\omega_i = \frac{1}{70}$ (relative to the dummy it has the same ranking, since the dummy is constant). The normalized score is secondary; the risk coordinates use a pooled-period exposure-weighted estimand (different weights, disclosed there). The physical score denominator $D_i = C_i^d - C_i^a$ has a heavy tail: min 2.4, 10th percentile 87, median 295, max 863; 8 of 70 sites have $D_i < 100$ and 4 have $D_i < 50$, where the score is very sensitive to small cost changes.

Controller	Score	Mean cost	Savings vs dummy
S_{AR} (SDP-AR(1) baseline)	0.7677	2235.7	261.0
R_P (actual-state persistence)	0.7442	2243.4	253.3
S_{AR} + physical filter	0.7677	2235.7	261.0
R_{FE96} (forecast-error rollout)	0.5794	2300.8	195.9
S_{AR} w/ forecast state	0.5689	2309.9	186.8
Behavior clone (per-site MLP)	0.5731	—	—

All controllers are causal (Section 4.11) and physically constrained (strict projection, zero infeasible actions). S_{AR} leads; R_P is the strongest proposed non-baseline rollout. The forecast-state controllers’ underperformance is **consistent with** the lower predictive quality of the stale day-ahead forecast, but the experiments do not isolate forecast quality from the selector and state-model differences (the two groups differ in state semantics, transition model, error quantization, and selector form).

Risk coordinates. Pooling per-period costs over all sites ($n = 2474$; this is an **exposure-weighted portfolio estimand** — large-cost and many-period sites dominate the tail; a per-site CVaR aggregated by equal weights is a different estimand and is not reported): S_{AR} achieves the best mean and tail risk ($E[J] = 1208.9$, $CVaR_{0.9}[J] = 15920.3$), R_P is close ($E[J] = 1216.9$, $CVaR_{0.9}[J] = 15928.2$), and R_{FE96} is clearly worst ($E[J] = 1273.4$, $CVaR_{0.9}[J] = 15968.8$). The CVaR-weighted objective of R_{FE96} does not buy tail-risk reduction here. Unit note: the leaderboard mean cost (2235.7) is the per-site mean over the whole test horizon, while $E[J]$ here is the per-period pooled mean; the two use different aggregation units and are not directly comparable.

Persistence attribution. The hypothesis that R_P wins where persistence is close to the AR model was tested per site: $\text{corr}(\delta_i^{R^2}, \delta_i^S) = -0.095$ (70 sites), i.e. no support; the selector-structure explanation remains a conjecture, not an established cause.

1.5.10 4.10 Automatic Parameter Optimization (OpenEvolve)

We ran the OpenEvolve evolutionary framework (DeepSeek LLM backend) to optimize the rollout’s hyperparameters under the physical convention. The fitness was the mean physical score on sites 1–3; the LLM evolved the `choose_parameters` logic over 5 iterations with a population of 6 (30 simulations, 75 min). The fitness progression and the 70-site validation of the best program:

Configuration	3-site fitness	70-site score	Note
Initial (0.25, 0.9, 0.5)	−0.1104	0.5794	—
Best evolved (0.03, 0.55, 0.25)	0.0352	0.5838	+0.0044
SDP-AR(1) baseline	0.5526	0.7677	reference

The optimizer worked (3-site fitness improved by 0.146), but the 70-site improvement is marginal (+0.0044) and no site beats the baseline. Within the searched three-parameter family and limited exploratory budget, hyperparameter changes did not close the structural gap to the baseline; the value-function physics is not the cause (Section 4.5: the value function is already energy-conserving). The selector structure and state semantics are a plausible explanation, stated as an inference rather than a proven unique cause. These runs are an **exploratory** evaluation: the 3-site fitness set is a subset of the 70-site report set, so the final set has seen development feedback and cannot serve as an independent validation.

1.5.11 4.11 Decision Timestamp and Causality (Code-Level Proof)

The decision time of step t is the timestamp of row $t + 96$. This is proven from four independent pieces of evidence:

1. **The EMSx paper’s information structure.** The paper states that “at the beginning of the time interval $[t, t + 1[$ we may use all the past observations”, defined as the partial observations $(w_t, w_{t-1}, \dots, w_{t-95})$ — the **current** realized demand plus the past 95 steps. `Information(t)` exposes exactly 96 actual values (rows $t + 1..t + 96$). These match $(w_{t-95}, \dots, w_t)$ only if w_t = the actual at row $t + 96$; under the alternative ($t_{\text{decision}} = \text{row } t$) the window would be 96 **future** demands, which contradicts the paper.
2. **apply_control settles at row $t + 97$** — under $t_{\text{decision}} = \text{row } t + 96$, the settlement is the **next** 15-minute interval (the controlled outcome, normal closed-loop semantics); under row t it would be 24 h 15 min later, which has no physical meaning.
3. **The original SDP uses $z_t = \text{load}[1] - \text{pv}[1]$ as the current net demand**, and `load[1]` is the actual at row $t + 96$; only the row- $t + 96$ interpretation makes this the current observation.
4. **The horizon is $\text{nrow} - 96$** ; the last decision (step $\text{nrow} - 96$) then falls at the last row, consistent with the window $t+1:t+96$. The settlement formula is `min(t + 96 + 1, horizon + 96)`; for the last two steps ($t = \text{horizon} - 1$ and $t = \text{horizon}$) this clamps to `nrow`, so the settlement row never exceeds the data range (site 1: $\text{horizon} = 17568$, last decision row $17664 = \text{nrow}$, settlement clamps to 17664). **Boundary caveat:** for the final step the settlement row equals the decision row, so the last action of every period is settled against the same actual as its own decision — a clamped convention inherited from the official simulator. Its cost share is about 1% of the total (R_P 1.021%, S_AR 1.034%, R_FE96 1.057%, 2474 periods); the timestamp contract is therefore stated as **interior steps timestamp-verified; the terminal boundary uses the clamped convention**. A corrected horizon ($H = \text{nrow} - 97$) or dropping the final step would remove it; this is recorded, not re-run here.

Runtime assertions at the decision time of step 1 (site 1, $t_{\text{decision}} = 2014-07-21\ 00:00$):

Assertion	Value	Holds
Latest actual (<code>load[1]</code> , row $t+96$) $\leq t_{\text{decision}}$	2014-07-21 00:00	yes
Forecast issue (row $t+1$) $\leq t_{\text{decision}}$	2014-07-20 00:15	yes
Settlement (row $t+97$) $> t_{\text{decision}}$	2014-07-21 00:15	yes
Settlement offset	15 minutes	—

Consequences: `load[1]` (row $t + 96$) is the **current** observation w_t — causal; the forecast origin (row $t + 1$) is a **day-ahead** forecast issued 95 steps earlier — causal; the settlement row $t + 97$ is the next interval, unknown at decision time. Every controller evaluated in this paper — the SDP-AR(1) baseline, R_P , and the forecast-error rollout — is causal (uses only actuals up to the decision time and forecasts issued before it). There is no lookahead quadrant; the deployment-target quadrant (causal information + physical dynamics) is exactly the physical leaderboard of Section 4.9. The previously reported L1/L2 split and the “causal baseline” experiment are re-interpreted in Section 4.12: the gap between the baseline (0.768) and the forecast-state SDP selector (0.569) is a **state-semantics mismatch** (feeding a settlement-forecast value into a value function whose z -coordinate means the current actual), not an information-level difference.

Forecast admissibility: three distinct notions. Temporal causality, Information-API admissibility, and the official benchmark protocol must not be conflated:

1. **Temporal causality** ($t_{\text{issue}} \leq t_{\text{decision}}$): the forecast issued at row $t + 96$ satisfies this (its issue time equals the decision time), so accessing it is **not** a temporal information leak under the reconstructed decision timestamp.
2. **Information-API admissibility:** the official constructor exposes the forecast of row $t + 1$ only (`data[end, ...]` in the original code); the row- $t + 96$ forecast is not present in the `Information` object. Reading it would require bypassing the interface — a **benchmark interface violation**.
3. **Official benchmark protocol:** the interface is the protocol’s information set; a controller may only use what the interface provides.

The earlier “96-step leak” terminology is therefore replaced: the fix restored the forecast origin exposed by the official Information interface. The row- $t + 96$ (current-time) forecast **exists in the raw data** (row i holds the forecast issued at row i), but it is not exposed by the official Information interface; its value under a separate deployment-oriented information protocol is not evaluated here.

Two information tracks. The official benchmark track uses only $\mathcal{I}_t^{\text{EMsX}}$ (the interface-exposed day-ahead forecast). A deployment track would allow the latest current-time forecast $\hat{z}_{t,t+1}^{\text{latest}}$; $\mathcal{I}_t^{\text{EMsX}} \subset \mathcal{I}_t^{\text{deployment}}$. Both are causal, but results from the two tracks must not be mixed into one leaderboard; the deployment track is a candidate route, not evaluated here.

1.5.12 4.12 Controller Naming and the Forecast-State Experiment

Three controllers are evaluated; fixed names throughout:

1. S_{AR} : the original stochastic SDP selector (the SDP-AR(1) baseline), state $z_t = \text{load}[1] - \text{pv}[1]$ (the current actual).
2. R_P : the actual-state persistence rollout (R_P), one-scenario $z = z_t$ (current actual), physical action filter.
3. R_{FE96} : the multi-scenario forecast-error rollout, baseline $b_t =$ the settlement-row day-ahead forecast (load_95 at row $t + 1$), physical action filter.

The experiment previously labeled “causal baseline” (the SDP selector fed with the settlement forecast b_t instead of the current actual z_t) is a **state-semantics ablation**, not an information-level experiment: S_{AR} with its z -state replaced by a forecast value scores 0.569, because the value function’s z -coordinate means the current actual, while the fed value is a next-interval forecast. The physical leaderboard (Section 4.9) is therefore entirely within the causal + physical deployment quadrant:

Controller	State	Score
S_{AR}	current actual z_t	0.7677
R_P	current actual (persistence)	0.7442
R_{FE96}	settlement forecast b_t	0.5794
S_{AR} w/ forecast state	settlement forecast b_t	0.5689

S_{AR} is the strongest controller; R_P is the strongest proposed non-baseline rollout. The forecast-state variants underperform, consistent with the lower predictive quality of the stale day-ahead forecast ($R^2 = 0.77$ vs 0.95 for the current actual); the experiments do not isolate forecast quality from selector and state-model differences.

1.6 5. Formal Verification with Lean

Selected algebraic properties of the finite-benchmark score and resampling summaries are machine-checked with Lean (Lean 4.32.2, mathlib v4.33.0-rc1, Leanproof/Reliability.lean; build green, no sorry/axiom/admit). The theorems are abstract and convention-independent.

1.6.1 5.1 Theorem 1: Score in $[0,1]$

Theorem 1 (score_mem_Icc).

If $m \geq a$, $0 \leq d - m$, and $0 < d - a$, then $\frac{d-m}{d-a} \in [0, 1]$.

This is a conditional statement; it does not assert that a causal controller satisfies $m \geq a$. Whether $C^m \geq C^a$ holds for a given oracle is an empirical question — and the environment-consistent oracle does not guarantee it (the score slightly exceeds 1 on 21 sites; this arises from numerical and oracle-approximation effects — the 21-point grid, nearest-neighbor interpolation and per-period $V_T = 0$ terminal mismatch — no exact lower-bound guarantee is claimed). The physical comparison is convention-dependent.

1.6.2 5.2 Theorem 2: Bootstrap LCB Bounded by Site Minimum

Theorem 2 (resample_mean_ge_min').

Every resample mean of the observed site scores is at least the site minimum.

Empirical fact about the 70 observed scores; not a population or continuous-operation confidence statement.

1.6.3 5.3 Theorem 3: Paired LCB Positive

Theorem 3 (paired_resample_mean_ge_min').

If the candidate beats the baseline on every site and the minimum paired gain is positive, then every paired resample mean is positive.

Holds only under the convention where the candidate wins all 70 sites (environment-consistent track). It does not hold under the physical convention, where the baseline wins on average.

1.6.4 5.4 Theorem 4: Infimum of Achievable Costs Is a Lower Bound

Theorem 4 (dpValue_lower_bound').

The infimum of achievable total costs is at most the total cost of every control sequence (when the set is bounded below).

This justifies the score denominator only when the oracle actually computes that infimum. The environment-consistent DP and the physical LP each solve well-defined (but different) optimization problems; Lean does not connect either oracle to its numerical implementation.

1.7 6. Limitations and Audit Responses

1. **Statistical scope.** LCBs are empirical site bootstrap bounds over the 70 observed sites; not population or future-operation inference. “All endpoints pass” is reworded to “the prespecified finite-benchmark empirical criteria are satisfied” and only in the environment-consistent track.
2. **Score conventions.** Three conventions exist and are never mixed in this paper: environment-consistent DP (candidate 0.9973, baseline 0.0746), physical LP (candidate 0.579, baseline 0.768), and official EMSx.evaluate_model (uses a reset-SOC anticipative baseline). The paper’s 0.794 threshold belongs to the official convention and is not used elsewhere.
3. **Oracle approximation and verification.** The environment-consistent DP uses a 21-point SOC grid with nearest-neighbor interpolation and per-period $V_T = 0$; it is an approximation, not a proven exact optimum. The physical LP oracle was verified by action replay: feeding the LP actions into the same simulator reproduces the LP objective ($|J_{LP} - J_{replay}| \leq 4.2 \cdot 10^{-11}$ per site, max SOC deviation $\leq 3.1 \cdot 10^{-16}$), confirming price/efficiency/unit/index consistency. The unbounded-SOC LP reproduces the environment DP’s objective on the tested instances, but it is **numerically equal**, not a semantically equivalent model of the clamped dynamics (the clamp keeps SOC at 0 while the unbounded LP lets it drift negative; the exploit policy makes the objective insensitive to SOC, which explains the agreement). The terminal convention is $V_T = 0$ per period with free terminal SOC and SOC carry across periods — explicit, finite-test horizon; the score denominator depends on it.
4. **Environment looseness.** The empty-battery discharge credit is an energy-conservation violation of the original benchmark. Both tracks are reported; the physical track is the usable one for deployment.
5. **Clamping and feasibility.** Decision-internal clamping is negligible (max $1.1 \cdot 10^{-3}$, median 0); the billed cost is never clamped. The physical rollout uses a strict closed-interval projection onto $U(x) = \left[\max\left(-1, -\eta_d C \frac{x}{|P|\Delta t}\right), \min\left(1, \frac{C(1-x)}{\eta_c |P|\Delta t}\right) \right]$ (the control grid lies within $[-1, 1]$, so the power bounds are implicit); with SOC carried across periods the infeasible-action rate is exactly zero.
6. **Continuous operation.** The evaluation is continuous-SOC finite-test-horizon with a periodic-average-cost value function approximation; not a proven infinite-horizon optimum.
7. **Parameter selection.** The operating point (k, lambda, alpha) was fixed by an earlier exploration on test sites 1–3 under the leaky semantics; it is a fixed heuristic operating point, not an unbiased-optimized configuration. OpenEvolve runs are exploratory.
8. **Causality.** All evaluated controllers are causal: the decision time of step t is the timestamp of row $t + 96$ (code-level proof in Section 4.11), so load[1] is the current observation and the forecast origin (row $t + 1$) is an already-issued day-ahead forecast. The physical leaderboard is the deployment-target quadrant; no lookahead controller exists in this evaluation.

9. **Determinism.** The forecast-error law is built deterministically (per-group weighted quantiles with fixed k and pseudocount; no random initialization), so identical inputs produce identical laws; the error-law cache records per-site SHA-256.

1.8 7. Conclusion

We audited the EMSx real-time evaluation and quantified a dominant confound. The settlement alignment of the forecast index is timestamp-proven (index 96 targets the settlement row exactly), and the forecast origin exposed by the official Information interface (row $t + 1$) is restored (the row- $t + 96$ forecast is not temporally noncausal under the reconstructed decision time but is not exposed by the interface). The benchmark environment credits discharge energy from an empty battery; the perfect-prediction oracle and the unconstrained candidate exploit this on essentially every step, and the environment-consistent score of 0.997348 is a measure of exploit parity, not of control quality.

Under a physically consistent action set and an independent physical LP oracle (verified by action replay), we established the physical-convention leaderboard with raw cost improvement as the primary metric. The strongest controller is the SDP-AR(1) baseline S_{AR} (score 0.768, savings 261.0 over the dummy); the strongest proposed non-baseline rollout is R_P (score 0.744, savings 253.3, 11/70 above the baseline, no negative-score sites). Both use the current actual at row $t + 96$ as their state, which the code-level proof (Section 4.11) shows is causal (decision time = row $t + 96$). The forecast-error rollout R_{FE96} (0.579) and the forecast-state SDP selector (0.569) underperform, which is consistent with the lower predictive quality of the stale day-ahead forecast ($R^2 = 0.77$ vs 0.95); the experiments do not isolate forecast quality from selector and state-model differences. A behavior-clone of the physical LP oracle (per-site MLP) reaches 0.573 and does not beat the model-based controllers: the oracle’s bang-bang optimal actions and flat optima are poorly captured by a single-step regressor. The audits, the timestamp contract, the exploit metrics, the physical oracle, and the two-track comparison are reproducible from this repository; selected algebraic properties of the finite-benchmark score and resampling summaries are machine-checked with Lean.

1.9 References

- [1] A. Le Franc, et al., “EMSx: a numerical benchmark for energy management systems”, paper/emsx_hal_v3.tex in this repository (the original EMSx.jl reference implementation).
- [2] Lean 4 + mathlib (Lean 4.32.2, mathlib v4.33.0-rc1): <https://leanprover-community.github.io>
- [3] HiGHS linear programming solver (via scipy; Julia 1.12.6, Python 3.14.6, scipy 1.18.0): <https://highs.dev>
- [4] Repository commits: audit worktree 1ab0115, EMSx.jl 3467500 (leak-free Information fix), report 92fa399; dataset at dataset/test in the worktree (regenerated by download scripts).

End of document